
Rapport de fin de stage

Un copilote d'intelligence artificielle pour le contrôle du trafic aérien

Parler aux avions, et que les avions répondent : choix scientifiques,
obstacles d'ingénierie et validation expérimentale de la chaîne vocale STT → LLM → TTS

Auteur	Nicolas Marano
Projet	Simulateur de contrôleur aérien (PoC, 10 semaines)
Établissement	Université de Reims Champagne-Ardenne (URCA)
Calcul (IA)	Supercalculateur ROMEO : NVIDIA GH200 (Grace-Hopper, aarch64)
Domaine	Intelligence artificielle & contrôle du trafic aérien
Dépôt	github.com/Gotman08/simulateur-controleur-aerien

Résumé. L'objectif premier de ce stage tient en une phrase : permettre à un élève contrôleur de parler à des avions, et que les avions lui répondent. Tout le reste en découle : pour éprouver ce dialogue dans des conditions réalistes, il fallait construire l'environnement complet qui l'entoure, un simulateur d'entraînement au contrôle aérien dont la boucle radio est entièrement pilotée par la voix. La clairance parlée de l'élève est transcrite (Whisper, que j'ai adapté au domaine par LoRA), interprétée en ordres structurés par un grand modèle de langage ancré dans une base de connaissances OACI, filtrée par des gardes déterministes, exécutée dans le simulateur de trafic BlueSky ; l'avion répond alors par un collationnement prononcé dans une voix de synthèse dégradée canal radio VHF. Dans ce rapport final, je justifie mes choix scientifiques au regard de l'état de l'art (pourquoi ces corpus, ces modèles, ces articles plutôt que d'autres), je raconte les difficultés rencontrées et la manière dont je les ai résolues, et j'apporte la preuve mesurée de ce qui fonctionne, et de ce qui ne fonctionne pas. Chiffres clés : WER de 71,9 % → 28,5 % par adaptation LoRA sur audio VHF réel (gain significatif, bootstrap apparié) ; 81,9 % d'exactitude d'interprétation pour le meilleur LLM local sur 116 clairsances annotées (comparaison appariée de 4 modèles, tests de McNemar) ; 10^6 décisions de conflit sans désaccord avec une vérité numérique indépendante et $F_1 = 1,0$ face à BlueSky ; boucle vocale complète à 76 % de réussite pour 2,6 s de latence moyenne sur GPU grand public, confirmée par une contre-épreuve avec un locuteur humain réel (80 %). L'ensemble du banc de mesure est versionné, seedé et re-exécutable à l'identique.

Table des matières

1	Introduction : contexte, mission et démarche	2
1.1	Le problème métier	2
1.2	La mission et la chaîne cible	2
1.3	Organisation du rapport	2
2	État de l’art et justification des choix scientifiques	3
2.1	Corpus de parole ATC : pourquoi ATCO2, UWB-ATCC et ATCOSIM	3
2.2	Reconnaissance vocale : pourquoi Whisper, et pourquoi LoRA	4
2.3	Interprétation : pourquoi un LLM ancré et non affiné, gardé par du déterminisme	4
2.4	Simulateur : pourquoi BlueSky	5
2.5	Synthèse vocale : pourquoi le clonage <i>zero-shot</i> , puis Kokoro en local	6
2.6	Synthèse des décisions	6
3	Réalisation : comment le système a été construit	6
3.1	Chronologie des dix semaines	6
3.2	Architecture finale	7
3.3	Sûreté en couches	7
3.4	Méthodes de travail	8
4	Difficultés rencontrées et résolutions	8
4.1	Contraintes d’infrastructure du cluster (S4–S6)	8
4.2	Perte du message système par le gabarit de conversation Mistral (llama.cpp)	10
4.3	Défaut de la garde sémantique montée/descente	10
4.4	Audit systématique du dépôt (juillet 2026)	10
4.5	Indisponibilité de ROMEO et continuité de service	11
4.6	Défauts détectés par la campagne de validation	11
4.7	Anomalie Qwen2.5-3B et contrôle par une seconde quantification	11
5	Validation expérimentale	12
5.1	Géométrie de conflit et simulateur	12
5.2	Reconnaissance vocale (STT)	12
5.3	Interprétation des clairances : comparaison appariée de quatre LLM et d’un parseur	13
5.4	Synthèse vocale	14
5.5	La boucle vocale complète	15
5.6	Reproductibilité	17
6	Limites et résultats négatifs	17
7	Bilan	19

1 Introduction : contexte, mission et démarche

1.1 Le problème métier

Un élève contrôleur s'installe en salle de simulation. Face à lui, un scope radar et une dizaine d'étiquettes qui avancent en temps réel ; dans son casque, le grésillement d'une radio VHF et des pilotes qu'il doit séparer, séquencer et guider dans l'anglais codifié de l'OACI. Deux avions convergent : il a quelques dizaines de secondes pour percevoir le conflit, choisir une manœuvre et la transmettre sans ambiguïté. Pour que cette scène fonctionne, il faut aujourd'hui des *pseudo-pilotes* : des opérateurs humains qui, derrière la cloison, prononcent les collationnements et exécutent chaque instruction de l'élève dans le simulateur.

Ce facteur humain domine le coût d'exploitation des simulateurs de formation. De là vient l'objectif premier de mon stage, facile à énoncer, beaucoup moins à réaliser : que l'élève puisse parler aux avions, et que les avions lui répondent, sans opérateur derrière la cloison. Le rôle à remplacer est difficile pour les modèles de fondation : comprendre une clairance parlée, l'exécuter fidèlement, la collationner d'une voix réaliste, alors que la phraséologie OACI est un langage contraint, que le canal radio VHF (bande utile 300–3 400 Hz) dégrade fortement le signal, et que toute erreur d'interprétation exécutée dans le simulateur fausse l'exercice pédagogique. Derrière la boucle vocale de 2,6 secondes mesurée à la fin de ce rapport, il y a donc un outil concret, conçu pour des humains en formation.

1.2 La mission et la chaîne cible

Le cœur de ma mission, durant ces dix semaines (un PoC encadré par l'URCA), était donc ce dialogue vocal bidirectionnel : l'élève parle, l'avion manœuvre et répond. Mais un dialogue ne se teste pas dans le vide : pour que l'avion ait un ordre à exécuter et un état à collationner, il faut un trafic simulé, un espace aérien, un radar. C'est ce qui m'a conduit à construire et évaluer toute la chaîne suivante, avec les modèles d'IA hébergés sur le supercalculateur ROMEO (nœud GH200 Grace-Hopper, aarch64, 96 Go) et le simulateur sur un poste local :

voix contrôleur (VHF) → Whisper STT → NER + LLM ancré (base OACI, graphe secteur) → JSON
validé
→ simulateur BlueSky (manœuvre) → collationnement en voix de synthèse VHF → boucle

Le livrable auquel j'ai abouti dépasse le PoC initial : une application web d'entraînement complète (scope radar 60 fps, strips, poste instructeur en langage naturel, mode exercice noté avec débriefing, figure 1), adossée à une campagne de validation mathématique et empirique et à un banc de mesure scientifique versionné avec le code. Il faut toutefois garder la hiérarchie en tête : radar, exercices et simulateur ne sont que l'infrastructure de test ; tout le projet se joue dans l'échange radio entre l'élève et l'avion.

1.3 Organisation du rapport

J'ai organisé ce document autour de trois questions : **(i)** pourquoi mes choix scientifiques (articles de référence, corpus, modèles, simulateur) plutôt que leurs alternatives (section 2) ; **(ii)** quelles difficultés j'ai rencontrées et comment je les ai résolues, preuves à l'appui (section 4) ; **(iii)** ce que je considère comme *prouvé*, avec quelles limites documentées (sections 5 et 6). La section 3 décrit au préalable comment j'ai construit le système, phase par phase.

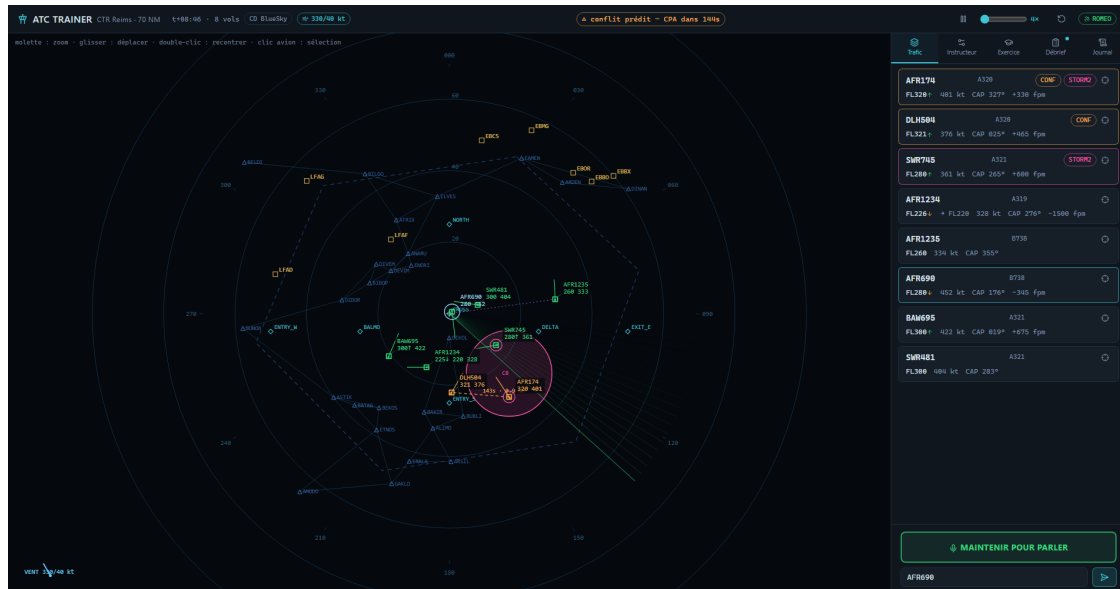


Figure 1 – L’application d’entraînement livrée : scope radar temps réel (BlueSky), strips de vol, poste instructeur et position contrôleur *push-to-talk*.

2 État de l’art et justification des choix scientifiques

Démarche de sélection bibliographique

J’ai constitué l’état de l’art en semaines 1 et 2 (`src/references.bib`) selon quatre critères explicites : **(a)** priorité aux travaux s’appuyant sur des *données radio réelles* plutôt que sur des données de laboratoire ; **(b)** reproductibilité (corpus publics, poids ouverts, code disponible) ; **(c)** compatibilité avec les contraintes matérielles du stage (cluster aarch64, quota disque de 20 Go par partition, 10 semaines, exécution locale possible) ; **(d)** actualité (2022–2026, le domaine évoluant très vite). Chaque choix retenu ci-dessous est confronté à ses alternatives, puis *validé a posteriori par une mesure*. C’est le fil conducteur de mon travail : je n’ai laissé aucune décision d’architecture reposer sur la seule autorité d’un article.

2.1 Corpus de parole ATC : pourquoi ATCO2, UWB-ATCC et ATCOSIM

Le choix des données d’entraînement conditionne tout l’étage de reconnaissance vocale. Avant de retenir les corpus de la communauté ATCO2, j’ai examiné puis écarté trois familles d’alternatives. La collecte directe de flux LiveATC fournit du volume, mais *sans transcriptions* et avec un statut juridique incertain ; inutilisable telle quelle pour un apprentissage supervisé en 10 semaines. Les corpus propriétaires des prestataires de navigation aérienne sont de meilleure qualité, mais inaccessibles dans le cadre du stage et non reproductibles par un tiers. Restaient les données synthétiques (TTS), disponibles à volonté, mais qui introduisent un écart de domaine (*sim-to-real*) précisément là où le projet cherche de la robustesse au canal réel.

Le triplet que j’ai retenu est issu des travaux de la communauté ATCO2 [1, 2, 3], qui a établi la reconnaissance vocale ATC comme domaine de recherche à part entière et publié les seuls grands corpus *réels, transcrits et publics* du domaine. Leur complémentarité a dicté mon protocole : UWB-ATCC [2] (radio réelle, bruitée, accents variés) et ATCOSIM [3] (parole de simulateur, propre) servent à l’*entraînement* et à la *validation*, le modèle voyant à la fois du canal dégradé et de la parole nette ; ATCO2-1h [1], l’audio le plus difficile (radio opérationnelle réelle), est *réservé au test* et n’est jamais vu à l’entraînement. Ce choix d’un jeu de test sévère et hors distribution est délibéré : je voulais mesurer la généralisation, pas la mémorisation.

Preuve a posteriori. Cette stratégie est validée par les mesures : le WER sur ATCO2 (domaine

jamais vu) passe de 71,9 % à 28,5 % après adaptation (section 5.2), et l'écart entre le WER de validation (6,68 %, domaine d'entraînement) et le WER de test (28–29 %) quantifie exactement le fossé de généralisation que le protocole était conçu pour révéler.

2.2 Reconnaissance vocale : pourquoi Whisper, et pourquoi LoRA

Whisper plutôt qu'une autre famille acoustique. L'intuition d'abord : inutile d'apprendre à un modèle à écouter depuis zéro quand il en existe un qui a déjà entendu des centaines de milliers d'heures de parole dans des conditions très variées ; il suffit de le spécialiser sur les nôtres. J'ai donc préféré Whisper [4] aux encodeurs auto-supervisés de type wav2vec2 et aux API de transcription commerciales, pour quatre raisons convergentes : (i) son entraînement faiblement supervisé à très grande échelle lui confère une robustesse au bruit documentée, déterminante sur un canal VHF ; (ii) ses poids sont ouverts, condition de l'adaptation au domaine et de la reproductibilité ; (iii) il est multilingue (le projet accepte la phraséologie anglaise *et* française) ; (iv) surtout, deux travaux récents démontrent spécifiquement sa transférabilité au contrôle aérien : la thèse de van Doorn [6] (TU Delft, 2023) établit la faisabilité de l'application de Whisper à l'ATC, et Su et Haq [7] (2026) fournissent un pipeline *économique en données* directement aligné avec la situation du stage (petit corpus, modèle `small`, canal réel). Ce dernier article s'est révélé le plus opérationnel des deux : j'en ai repris le front-end de canal, un passe-bande de Butterworth 300–3 400 Hz appliqué à l'entraînement *et* à l'inférence (autrement dit, faire entendre au modèle, dès l'apprentissage, le même son étouffé qu'une vraie radio), ainsi qu'une mise en garde d'implémentation sur le couple LoRA / `input_features` qui m'a évité une erreur classique (Whisper est un modèle séquence-à-séquence dont l'encodeur consomme des spectrogrammes, pas des identifiants de tokens : le collateur doit rembourrer séparément features et labels).

Une API commerciale aurait par ailleurs interdit l'adaptation au domaine, imposé une dépendance réseau au poste de formation et fait sortir les données ; ces trois points sont rédhibitoires pour un simulateur d'entraînement déployable hors ligne.

Taille `small`. `whisper-small` (244 M de paramètres) est le compromis que j'ai retenu entre qualité et coût : il tient largement sur le GH200 pour l'entraînement, et son facteur temps réel d'inférence mesuré (RTF 0,04 sur cluster, 0,017 sur GPU portable) laisse une marge confortable pour la boucle interactive. Les tailles supérieures auraient allongé les itérations d'entraînement sans lever le goulot réel, la qualité des données du domaine, et les mesures finales confirment que l'adaptation domine l'effet de taille : le `small` adapté bat très largement des modèles génériques de même gabarit (section 5.2).

LoRA plutôt qu'un affinage complet. L'idée de LoRA [5] : au lieu de réécrire le livre, on l'annote dans la marge. Le modèle de base est gelé, et je n'entraîne que de petites matrices correctrices de rang faible ($r = 32$, $\alpha = 64$), insérées dans les projections d'attention `q_proj/v_proj` de l'encodeur et du décodeur. Réentraîner les 244 M de paramètres de `whisper-small` aurait été instable sur un corpus modeste, et incompatible avec le quota disque de 20 Go du cluster (chaque exécution aurait produit des points de contrôle de plusieurs Go) ; l'adaptateur LoRA, lui, pèse 14 Mo (144 tenseurs), se versionne dans le dépôt git, se distribue et se reverse trivialement. C'est l'approche aujourd'hui dominante pour l'adaptation de Whisper au domaine ATC [7, 6].

2.3 Interprétation : pourquoi un LLM ancré et non affiné, gardé par du déterminisme

De tous mes choix d'architecture, c'est le plus structurant. Le principe : au lieu d'apprendre la phraséologie au modèle, je lui mets la documentation sous les yeux au moment de répondre, et je fais vérifier chacune de ses réponses par du code classique qui, lui, ne se trompe jamais sur une borne numérique. Ce choix s'appuie sur une lecture *critique* de trois articles complémentaires.

Zhang et Mott [8] (2024) évaluent les LLM sur la reconstruction de trajectoires de vol et concluent à leur *faiblesse sur les longues séquences de coordonnées numériques brutes*. Cet article m'a servi de contre-exemple fondateur : il proscriit d'injecter l'état brut du trafic (positions, vitesses) dans le prompt. Je fournis à la place au modèle une représentation *symbolique compacte* de l'espace aérien, un graphe de secteur (7 points, 6 segments, séparation 5 NM) résumé en trois lignes de texte, et je laisse tout le calcul géométrique à du code déterministe.

AirTrafficGen [9] (2025) démontre l'efficacité d'une représentation de l'espace aérien *sous forme de graphe* pour la génération de scénarios de trafic par LLM ; il a directement inspiré le module `graph_secteur` (nœuds typés entrée/fixe/sortie, arêtes pondérées en NM, Dijkstra pour le routage) et la fonctionnalité « poste instructeur » où une description en langage naturel devient une situation de trafic.

Sharifi et al. [10] (2026), enfin, affinent des LLM (SFT puis DPO) pour la déconflition tactique de drones. J'ai étudié cette voie, *spécialiser le LLM lui-même*, puis je l'ai écartée, pour trois raisons : il n'existe pas de corpus public appariant clairances ATC et commandes exécutables à l'échelle requise par un affinage ; le coût d'itération (10 semaines, quota disque) était incompatible ; enfin, dans un domaine où une erreur exécutée fausse l'exercice, je refusais de faire reposer la sécurité sur des poids appris, par nature non auditables. J'ai donc retenu l'architecture inverse : un modèle généraliste *ancré* (mes 9 fiches de phraséologie inlinées dans le prompt, des indices NER, le graphe de secteur) dont *chaque* sortie traverse une validation déterministe et testée (bornes physiques HDG 0–360° / ALT 0–45 000 ft / SPD 0–350 kt, actions autorisées, waypoints du secteur, cohérence montée/descente).

Le cas particulier du Doc 4444. La source normative de la phraséologie est le Doc 4444 de l'OACI [15]. Ce document étant protégé, je ne pouvais ni l'ingérer ni le citer verbatim dans une base RAG versionnée : j'ai donc rédigé une base de connaissances *originale* de 9 fiches factuelles (~2,5 Ko) qui synthétisent les règles (conversions de chiffres, indicatifs en téléphonie, contextes informatifs à ignorer comme le QNH, règle « en cas de doute, ne rien produire ») en intégrant les bornes déjà codées du connecteur BlueSky. La recherche s'effectue par embeddings `bge-small-en-v1.5` et similarité cosinus en NumPy pur : un index FAISS aurait ajouté une dépendance lourde sans bénéfice mesurable sur une base de 9 documents.

Preuve a posteriori. La pertinence de l'approche « généraliste ancré + gardes » est démontrée par la campagne comparative (section 5.3) : le meilleur LLM local atteint 81,9 % d'exactitude stricte, *bat le parseur à règles sur les formulations hors grammaire* (77,1 % contre 70,8 %), ce qui est précisément sa raison d'être ; et sur les 116 clairances du banc, aucun ordre hors bornes n'a atteint le simulateur, quel que soit le modèle, y compris les plus faibles : la sécurité repose sur les gardes déterministes, non sur le modèle.

2.4 Simulateur : pourquoi BlueSky

BlueSky [11] (TU Delft) est le simulateur de trafic open source de référence de la recherche ATM. J'ai écarté les alternatives (simulateurs commerciaux fermés, plateformes de type Euroscope orientées contrôle en réseau, développement d'un moteur maison) respectivement pour inaccessibilité, inadaptation au pilotage programmatique, et coût hors de portée d'un PoC. BlueSky offre exactement ce dont ma boucle a besoin : un mode *headless* pilotable par commandes texte (TrafScript), des modèles de performances avion réalistes (OpenAP), une détection de conflits native (StateBased), et une base de code Python inspectable. **Preuve a posteriori** : le simulateur avance 10 s de trafic avec un facteur temps réel de $\times 105$ à 5 aéronefs et encore $\times 67$ à 200 aéronefs (figure 5), il n'est donc jamais le facteur limitant, et son détecteur de conflits confronté au prédictif analytique du projet donne une matrice de confusion parfaite sur 200 rencontres (section 5.1).

2.5 Synthèse vocale : pourquoi le clonage *zero-shot*, puis Kokoro en local

Mon cahier des charges imposait une voix *distincte et stable par aéronef*. Entraîner un modèle TTS par locuteur était exclu (ni corpus dédié, ni budget de calcul, ni quota disque) ; j’ai donc retenu XTTS-v2 [14] pour sa capacité de clonage *zero-shot* : un extrait de référence de quelques secondes (tiré du corpus ATCO2, donc au timbre « radio » authentique) suffit à produire une voix, sans aucun entraînement. Pour le point de déploiement local, j’ai ensuite ajouté le moteur Kokoro-82M (poids ouverts, ONNX) : pas de clonage, mais un pool de voix stables (affectation par hachage CRC32 de l’indicatif) et un facteur temps réel de 0,34 sur GPU portable. Dans les deux cas, la dégradation VHF (Butterworth d’ordre 6, 300–3 400 Hz) est appliquée *côté client*, de sorte que le réalisme radio et la cohérence de domaine avec l’ASR sont garantis quel que soit le fournisseur.

2.6 Synthèse des décisions

Décision retenue	Alternatives écartées	Critère déterminant	Preuve a posteriori
UWB+ATCOSIM (train), ATCO2 réservé au test	LiveATC sans transcription ; corpus propriétaires ; audio synthétique	données réelles transcrites ; test hors distribution	WER 28,5 % sur domaine jamais vu ; fossé 6,7 → 28,5 % quantifié
Whisper-small + LoRA ($r=32$)	affinage complet ; wav2vec2 ; STT cloud	robustesse bruit ; poids ouverts ; adaptateur 14 Mo (quota 20 Go)	gain WER significatif (bootstrap apparié, IC excluant 0)
LLM généraliste ancré + gardes déterministes	affinage SFT/DPO du LLM [10] ; règles seules	audibilité ; pas de corpus d’affinage ; sécurité prouvable	0 ordre hors bornes exécuté / 116 ; hors grammaire : LLM 77,1 % > parseur 70,8 %
Contexte spatial = graphe compact en texte	coordonnées brutes dans le prompt	LLM faibles sur séquences numériques [8]	100 % de JSON exploitable (40 transcriptions réelles)
BlueSky headless	simulateurs fermés ; moteur maison	open source ; TrafScript ; CD native	×67–105 temps réel ; $F_1=1,0$ vs prédicteur
XTTS-v2 zero-shot / Kokoro-82M local	TTS entraîné par locuteur ; TTS cloud	aucune donnée par voix ; hors ligne	voix par aéronef ; RTF 0,34 local ; WER aller-retour 21,4 %
Contrat OpenAI-compatible unique	double mode LOCAL/ROMEO à bascule automatique	pannes visibles ; interchangeabilité	continuité de service pendant la panne totale de ROMEO (juill. 2026)

Table 1 – Chaque décision structurante : les alternatives considérées, le critère déterminant, et la mesure de validation a posteriori.

3 Réalisation : comment le système a été construit

3.1 Chronologie des dix semaines

Le tableau 2 retrace les grandes phases du stage, du socle audio des premières semaines jusqu’à la campagne de mesure finale, avec pour chacune son livrable mesurable. On y lit la logique d’ensemble : construire un à un les maillons du dialogue élève-avion (entendre en S4, comprendre en S5, répondre en S6), les assembler (S7, S8), puis bâtir l’environnement d’entraînement et l’appareil de preuve qui permettent d’éprouver ce dialogue en conditions réalistes (juin et juillet). Les sous-sections suivantes décrivent l’architecture obtenue et les méthodes de travail transversales.

Période	Contenu	Livrable / mesure
S1–S3	état de l’art ; front-end audio VHF (Butterworth ordre 6) ; graphe de secteur ; connecteur JSON→TrafScript ; NER à règles	briques pures réutilisées partout
S4	affinage LoRA de <code>whisper-small</code> sur UWB-ATCC + ATCOSIM	WER ATCO2 74,3 → 29,2 % ; val. 6,68 %
S5	base OACI (9 fiches), retrieval, prompt strict, validation de sécurité	100 % JSON exploitable ; 5/5 adversariaux bloqués
S6	clonage vocal XTTS-v2 + dégradation VHF, collationnement OACI	3 voix clonées ; 3,9 s / collationnement
S7	requête unifiée : transcription + règles RAG + graphe secteur → JSON	4 couches de validation en aval
S8	boucle temps réel complète sur ROMEO (tunnel SSH, SLURM)	manœuvre sur instruction parlée vérifiée
11 juin	application d’entraînement (React + FastAPI), mode exercice noté, campagne de validation mathématique et empirique	$F_1=1,0$ vs BlueSky ; 110 tests
1–2 juill.	audit multi-agents (6 défauts traités) ; refonte API-first OpenAI-compatible ; validation stricte des entrées	148 puis 186 tests ; pannes visibles (502)
10–21 juill.	banc de mesure scientifique complet sur RTX 4070 ; durcissement production (revue : 32 constats corrigés) ; article de recherche	209 tests ; <code>bench/results/*.json</code> ; <code>docs/article/article.pdf</code>

Table 2 – Les grandes phases du stage. Chaque phase a fait l’objet d’un rapport hebdomadaire (`rapport_S4–S5`, `rapport_S6–S7`) dont ce document est la synthèse finale.

3.2 Architecture finale

La figure 2 présente l’architecture que j’ai livrée. Le navigateur (React, canvas radar, *push-to-talk*) communique avec un serveur FastAPI par REST et WebSocket (~ 8 Hz). Le simulateur BlueSky, non *thread-safe*, vit dans un thread unique alimenté par une file de commandes. Les trois briques d’IA sont consommées à travers le contrat REST OpenAI-compatible standard (`/v1/audio/transcriptions`, `/v1/chat/completions`, `/v1/audio/speech`), configurées par variables d’environnement : changer de fournisseur, qu’il s’agisse de la façade auto-hébergée sur ROMEO (modèles du projet), de la façade 100 % locale (llama.cpp / faster-whisper / Kokoro, fournie avec le banc) ou d’une API cloud, ne modifie ni le code ni les gardes de validation.

3.3 Sûreté en couches

Le contrat d’interprétation impose un tableau JSON d’ordres `{callsign, action ∈ {HDG, ALT, SPD, ADDWPT}, value}`. En aval du modèle, une bibliothèque pure et testée applique : coercition stricte de type (rejet des booléens, NaN, infinis, chaînes numériques), bornes physiques, validation des waypoints contre le graphe de secteur. J’y ajoute deux gardes applicatives : un indicatif absent du radar produit un silence radio (réalisme assumé), et une incohérence sémantique montée/descente (« climb » vers un niveau inférieur au niveau courant, typiquement causée par une transcription tronquée) est rejetée avec message. Enfin, aucune panne de fournisseur n’est rattrapée silencieusement : erreur typée → HTTP 502 → événement visible dans le journal de bord de l’interface. La qualité logicielle est portée par 209 tests unitaires (CI GitHub), une campagne de validation re-exécutable munie de portes de non-régression, et le banc de mesure de la section 5.

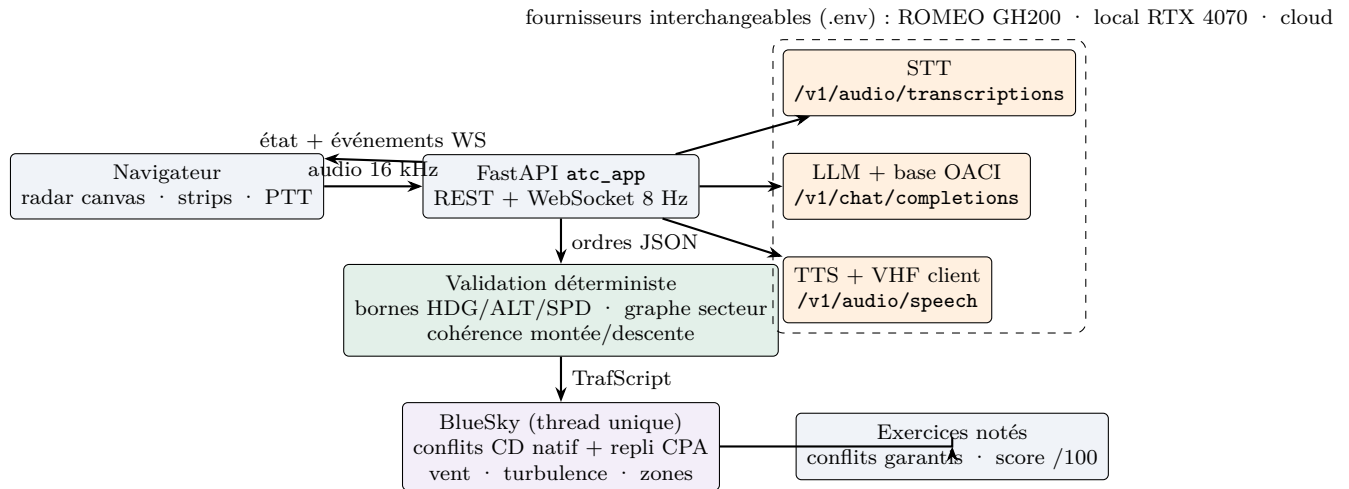


Figure 2 – Architecture livrée. Les briques d’IA (orange) sont des services OpenAI-compatibles interchangeables ; aucune sortie de LLM n’atteint le simulateur sans la couche de validation déterministe (vert).

3.4 Méthodes de travail

Trois pratiques m’ont accompagné tout au long du stage et ont produit une part importante des résultats : **(i)** j’ai toujours validé chaque brique *isolément* avant de l’assembler (qualité du retrieval avant branchement du LLM, test d’overfitting sur 16 extraits avant entraînement complet, baseline zero-shot avant affinage) ; **(ii)** j’ai fait passer toutes les campagnes de mesure par le *code de production* (mêmes prompts, mêmes clients HTTP, même post-traitement), jamais par une reconstitution ; **(iii)** j’ai mené la revue de code finale en adversarial (chaque constat devant d’abord résister à une tentative de réfutation sur le code réel), ce qui m’a conduit à confirmer 32 constats mais aussi à en *réfuter* 10, et à reclasser un « bug » de notation en comportement voulu, désormais figé par un test.

4 Difficultés rencontrées et résolutions

Cette section est probablement la plus fidèle à ce qu’ont réellement été ces dix semaines : une succession d’obstacles imprévus, de fausses pistes et de résolutions. Le tableau 3 en donne la vue d’ensemble factuelle ; je raconte ensuite les épisodes les plus instructifs tels qu’ils se sont déroulés. Pour chacun, la résolution est adossée à une *preuve* (mesure avant/après, test de non-régression, fichier de résultats versionné).

4.1 Contraintes d’infrastructure du cluster (S4–S6)

Travailler sur le nœud GH200 de ROMEO, une machine singulière (ARM aarch64 + Hopper, 96 Go unifiés), m’a appris qu’un environnement exotique se paie en incidents inattendus. En voici les principaux.

Quota disque de 20 Go. Ma première préparation du corpus s’est arrêtée net : `save_to_disk` et `.map` dupliquaient toute la table audio et faisaient déborder le quota de 20 Go par partition. Plutôt que de subir la contrainte, j’ai repensé la gestion des données : reconstruction du `DatasetDict` à la volée depuis le cache Hugging Face sans jamais matérialiser de copie, filtrage par sélection d’indices paresseuse, séparation des caches entre partitions (datasets et embeddings sur SCRATCH, poids Mistral sur PROJET). En semaine 6, la même contrainte m’a fait installer `coqui-tts` dans l’environnement existant plutôt que dans un venv séparé, pour éviter une seconde installation de PyTorch ; ce choix contraint a eu un bénéfice collatéral inattendu : les trois modèles (Whisper, Mistral, XTTS) ont fini servis par un unique processus.

Phase	Problème	Cause racine	Résolution	Preuve
S4	entraînement gelé, GPU à 0 %	workers DataLoader démarrés en fork après init. CUDA	passage à spawn + transformation picklable	débit GPU rétabli (test T4)
S4	échec de la génération en évaluation	conflit de dtype bf16 / fp32	entraînement bf16, génération forcée fp32	test d'overfitting, WER ≈ 0
S4	quota disque 20 Go dépassé	save_to_disk/.map dupliquent la table audio	dataset paresseux depuis le cache HF, filtrage par indices	3 époques complètes sous quota
S4	gradients non propagés (PEFT + checkpointing)	entrées sans requires_grad	enable_input_require_grads()	convergence de la perte
S5	Doc 4444 protégé (droit d'auteur)	impossible d'ingérer le manuel	base originale de 9 fiches synthétisées	retrieval validé isolément
S6	plantage cuFFT sur GPU (XTTS)	bug torchaudio/cuFFT sur GH200 aarch64	synthèse basculée sur CPU	3,9 s / collationnement (RTF 0,82)
S6	ImportError à l'import de XTTS	symbole isin_mps_friendly supprimé de transformers 5.x	correctif restaurant le symbole avant l'import (sans rétrograder)	Whisper + Mistral + XTTS servis par un même processus
S8	clairance « descend » tronquée exécutée en montée	hallucination du LLM sur texte incomplet	garde sémantique verbe/niveau	cas rejoué et bloqué (job 651579)
juin	repli anti-conflit = code mort	test <code>type(cd).__name__</code> face à un objet Proxy BlueSky, jamais vrai	drapeau d'état positionné à l'activation	audit B2; test dédié
juill.	bascule LOCAL/ROMEO silencieuse masquant les pannes	architecture double mode	mode unique API OpenAI-compatible; toute panne → 502 visible	panne ROMEO absorbée (§4.5)
juill.	HTTP 500 sur entrées malformées	types non coercés ("35000", True, flottants) acceptés	coercition stricte; payloads bornés (400/413/502)	tests adversariaux; suite à 209 tests
juill.	llama.dll introuvable (llama.cpp CUDA)	DLL CUDA absentes du PATH Windows	os.add_dll_directory vers les DLL du wheel torch	llm_bench.json : gpu_offload=true
juill.	Mistral-7B effondré à 20,7 % au banc	le gabarit de conversation llama.cpp perd le message système	fusion système → utilisateur (gabarit officiel Mistral)	20,7 → 81,9 % (§4.2)
juill.	garde sémantique contournable	comparaison de chaînes $32000.0 \neq 32000$	filtrage positionnel des listes ordres/lignes; valeur coercée	revue production; test de non-régression
juill.	durée de perte de séparation gonflée	ré-entrée d'une même paire recomptée depuis le début	épisodes cumulés	tests du barème de notation

Table 3 – Vue d'ensemble des problèmes rencontrés. S1–S8 : semaines de stage ; juin/juillet : phases d'industrialisation puis de campagne de mesure.

L'inter-blocage du DataLoader. Lors du premier entraînement complet, j'ai été confronté à un blocage total. La perte ne progressait plus ; le GPU tombait à 0 % ; aucune erreur, aucun message. Après avoir suspecté tour à tour mes données puis la configuration SLURM, j'ai fini par identifier la cause profonde : les workers du DataLoader démarraient en **fork** après l'initialisation de CUDA et du décodeur audio, un scénario de deadlock classique mais parfaitement silencieux. Forcer le démarrage en **spawn** a levé le blocage, au prix d'une conséquence en cascade : la transformation de données devait devenir picklable, ce qui m'a fait remplacer une closure par une classe (**WhisperTransform**). Cet épisode m'a montré qu'une solution d'infrastructure peut redessiner le code lui-même.

Le bug cuFFT et le correctif de compatibilité transformers. La synthèse vocale m'a opposé deux obstacles successifs. D'abord un plantage systématique de l'encodeur de locuteur de XTTS sur le GPU : un bug cuFFT de torchaudio propre à l'architecture Grace-Hopper, que je ne pouvais pas corriger moi-même. J'ai donc basculé la synthèse sur CPU, fiable mais lente (3,9 s par collationnement, RTF 0,82), et assumé ce compromis. Ensuite une **ImportError** : XTTS importe un symbole supprimé de transformers 5.x (**isin_mps_friendly**), alors que Whisper et Mistral m'imposaient de rester en version 5.9. Plutôt que de rétrograder, j'ai restauré le symbole par un

correctif appliqué avant l'import. Je documente ce correctif comme une *dette technique assumée* : il casserait silencieusement à une montée de version, et je préfère l'écrire noir sur blanc que le découvrir plus tard.

4.2 Perte du message système par le gabarit de conversation Mistral (llama.cpp)

Le 10 juillet, au premier passage du banc LLM, un résultat m'a fait douter de toute ma campagne : Mistral-7B, pourtant le modèle servi avec succès sur ROMEO depuis des semaines, s'effondrait à 20,7 % d'exactitude, avec 0 % de JSON strictement valide. Mon premier réflexe : suspecter la quantification Q4. Le deuxième : mon propre prompt. Pour trancher, j'ai écrit un test dédié qui inspecte ce que le moteur envoie réellement au modèle, et le verdict était ailleurs : le gabarit de conversation Mistral-v0.3 embarqué par llama.cpp *perd purement et simplement le message système*, donc tout l'ancrage OACI, le schéma JSON et les règles de sécurité de mon prompt. Le gabarit officiel de Mistral fusionne le message système dans le premier message utilisateur ; j'ai reproduit ce comportement par l'option `-merge-system` de ma façade locale. Résultat, versionné dans deux fichiers de mesure (`llm_bench_v1_avant_correctifs.json` et `llm_bench.json`) :

Mistral-7B-Instruct-v0.3 (Q4_K_M)	10 juill. (avant)	21 juill. (après)	
Exactitude TrafScript exacte (116 clairances)	20,7 %	81,9 %	×4
JSON strictement valide	0,0 %	100 %	
Erreurs fournisseur	2	0	

Table 4 – Effet du correctif de gabarit `-merge-system`. Les deux JSON de mesure sont versionnés dans `bench/results/`.

Ce cas m'a laissé une règle : *un modèle s'évalue à travers sa pile de déploiement réelle*, pas isolément. Le même poids, la même quantification et le même prompt donnaient 21 % ou 82 % selon un détail du moteur de service. De là vient la méthodologie « mesure à travers le code de production » appliquée à tout mon banc.

4.3 Défaut de la garde sémantique montée/descente

S'il fallait ne retenir qu'un épisode de ce stage, ce serait celui-ci, car il m'a fait vivre le cycle complet défaut, correctif, nouveau défaut. En semaine 8, un incident réel : une clairance « descend flight level one... » tronquée par l'ASR conduit le LLM à halluciner un niveau *supérieur* au niveau courant ; l'avion serait monté sur un ordre de descente. J'ai ajouté une garde comparant le verbe prononcé au sens effectif du changement de niveau, rejoué le cas sur le cluster (job 651579) et vérifié qu'il était bloqué. J'étais convaincu d'avoir fermé la porte. Un mois plus tard, la revue de production adversariale m'a détrompé : ma garde était contournable. Le filtrage reposait sur une comparaison de chaînes entre la valeur brute produite par le LLM (32000.0) et la valeur normalisée de la ligne TrafScript (32000) : en cas de désaccord de forme, l'ordre *rejeté et annoncé comme bloqué* pouvait tout de même être exécuté. J'ai remplacé ce mécanisme par un filtrage positionnel sur les listes parallèles ordres/lignes, et aligné la valeur de l'ordre sur l'entier effectivement coercé. J'en retiens deux enseignements : une garde de sécurité doit être vérifiée de manière adversariale, comme si l'on cherchait soi-même à la contourner ; et l'annonce d'un blocage doit être dérivée de l'action réellement appliquée, non d'un calcul parallèle.

4.4 Audit systématique du dépôt (juillet 2026)

Début juillet, j'ai retourné l'exigence de preuve contre mon propre code : un audit systématique de l'ensemble du dépôt (14 agents de relecture spécialisés, chaque verdict adossé à une preuve d'exécution). Six défauts numérotés en sont sortis, tous traités, dont un qui m'a particulièrement marqué, un *code mort de sécurité* (B2) : le repli géométrique de détection de conflits testait le type

de l'objet BlueSky par `type(cd).__name__ == "ConflictDetection"` alors que BlueSky expose un `Proxy` ; la condition n'était donc *jamais* vraie et le repli ne pouvait pas s'activer. Mon correctif remplace l'inspection de type par un drapeau d'état positionné au moment de l'activation. L'audit a aussi reclassé un constat en *non-bug* (la « double pénalité » supposée du barème est en réalité la sémantique documentée de deux compétences distinctes, désormais figée par un test) : il faut savoir distinguer ce qui est cassé de ce qui est simplement surprenant. La suite de tests est passée de 110 à 148 tests à l'issue de l'audit, puis à 209 après la revue de production.

4.5 Indisponibilité de ROMEO et continuité de service

L'audit avait identifié un risque structurel : l'architecture initiale à deux modes (LOCAL/ROMEO) pouvait *basculer silencieusement* d'un fournisseur à l'autre, masquant les pannes. Le 2 juillet, j'ai donc tout refondu en un mode unique au contrat OpenAI-compatible : trois URL dans un fichier `.env`, et toute panne remontée en HTTP 502 visible. Je pensais corriger un défaut de visibilité ; je préparais en réalité, sans le savoir, la survie de ma campagne de mesure. Trois semaines plus tard, au moment précis où je lançais les benchmarks, le supercalculateur ROMEO est devenu totalement indisponible (maintenance de sécurité, correctifs RedHat en attente sur les nœuds de connexion et de calcul). Privé du cluster, je n'ai pourtant rien eu à réécrire : une bascule du `.env` vers la façade 100 % locale (RTX 4070), et l'application comme le banc sont restés pleinement opérationnels. Une décision d'architecture prise pour des raisons de *visibilité des pannes* s'est trouvée validée par un incident réel de *continuité de service* qu'elle n'anticipait pas.

4.6 Défauts détectés par la campagne de validation

Ma campagne de validation (section 5) n'a pas seulement produit des chiffres : elle m'a renvoyé mes propres défauts, ce qui est exactement son rôle.

Le cas le plus parlant est celui des clairances françaises, ignorées en silence. À la première passe du parseur, j'obtenais 92,6 % global mais 1/6 sur la strate française : le découpage indicatif/instruction s'appuyait sur une liste de mots-clés uniquement anglais, et une phrase entièrement française, sans aucun déclencheur, était ignorée sans message. J'ai ajouté les verbes français et étendu les grammaires (*niveau de vol*, *cap*, *vitesse*...) ; résultat : 68/68 (100 %), sans régression sur la suite de tests. Même scénario pour les types d'aéronefs : « four B744 » produisait des A320, la regex ne reconnaissant pas B744 et la liste blanche étant incohérente (B744 sans B747, A380/A388 symétriquement). J'ai étendu la regex et introduit une table d'alias vers les variantes connues de la base de performances BlueSky, au lieu du repli silencieux sur A320 ; résultat : 18/18 sur les contraintes de type. Le banc a aussi montré que de vrais modèles produisent des nombres JSON à zéro de tête (`"value": 090`, invalide au sens strict) : l'analyseur tolérant du projet répare ce cas.

Ces trois cas illustrent la fonction de contrôle de la campagne de mesure : les pourcentages intermédiaires « non parfaits » (92,6 %, 94,4 %), je ne les ai pas publiés comme résultats ; je les ai exploités comme *détecteurs de défauts*, puis j'ai revalidé les correctifs par la même campagne.

4.7 Anomalie Qwen2.5-3B et contrôle par une seconde quantification

Le banc comparatif m'a réservé une surprise : Qwen2.5-3B s'effondrait à environ 24 % là où son cadet 1.5B atteignait 75 %, à rebours de l'intuition (l'échelle devrait aider). Ma première hypothèse était un fichier GGUF défectueux ; la seconde, un modèle qui ignorerait mon message système. L'investigation a écarté les deux, dans l'ordre inverse : le modèle honore bien le message système (je l'ai vérifié), mais viole le *schéma* demandé, champ `callsign` omis, clés JSON non citées. Restait l'hypothèse du fichier. Avant de publier un résultat aussi sévère, j'ai ré-évalué une seconde quantification indépendante (bartowski) : 23,3 %, statistiquement indiscernable de la première (McNemar, $p = 1,0$). J'ai donc publié ce résultat négatif accompagné de son expérience de contrôle : c'est le modèle qui est inadapté à la tâche contrainte, pas le fichier.

5 Validation expérimentale

Méthodologie du banc de mesure

J’ai mesuré chaque étage *à travers le code de production* (mêmes prompts, mêmes clients HTTP, même post-traitement), jamais par une reconstitution. Tous mes protocoles sont seedés et versionnés (**bench/**) ; chaque script écrit un JSON auto-porteur (protocole + résultats) dont les figures, tableaux et le README sont générés automatiquement : aucun chiffre publié n’est saisi à la main. Statistiques : IC à 95 % par bootstrap percentile ($B = 10^4$, graine fixée) pour les moyennes, IC de Wilson pour les proportions, test exact de McNemar pour les comparaisons appariées, bootstrap apparié pour les différences de WER. L’intuition du test apparié : plutôt que de comparer deux moyennes globales, on regarde clairance par clairance les cas où un système réussit là où l’autre échoue ; c’est ce qui permet de dire si un écart est réel ou s’il tient au hasard de l’échantillon. Matériel : campagne locale sur RTX 4070 Laptop 8 Go (i9-13900H, 32 Go) ; références historiques sur ROMEO GH200.

5.1 Géométrie de conflit et simulateur

Le prédicteur de conflit embarqué repose sur une observation géométrique simple : deux avions en ligne droite se rapprochent puis s’éloignent, et leur distance au carré décrit une parabole. Une parabole strictement convexe n’a qu’un seul creux ; l’instant du rapprochement maximal (CPA) se calcule donc par une forme close, sans simulation. En mouvement rectiligne uniforme, le coefficient dominant de $d^2(t)$ est $\|\vec{v}\|^2 > 0$, et $t^* = -\vec{r}_0 \cdot \vec{v} / \|\vec{v}\|^2$ est le minimum global unique. J’ai vérifié cette exactitude numériquement à trois niveaux.

Contre une vérité numérique indépendante. Sur 10^6 géométries Monte-Carlo (5 graines $\times$ 200 000 ; positions ± 80 NM, caps uniformes, vitesses 150–550 kt), la décision de conflit coïncide avec une recherche sur grille fine ($\Delta t = 0,05$ s) dans *tous* les cas : 0 désaccord sur 10^6 décisions (figure 3). L’erreur maximale sur d_{CPA} est bornée par l’arrondi d’affichage du champ comparé (0,05 NM) ; la campagne historique, qui comparait les valeurs non arrondies, mesurait une erreur maximale de $5,52 \times 10^{-4}$ NM (≈ 1 m).

Contre BlueSky. Sur 200 rencontres stabilisées (55 % biaisées conflit), précision = rappel = $F_1 = 1,0$ (113 conflits), MAE de 0,067 NM sur d_{CPA} et de 0,71 s sur t_{CPA} (figure 4). Ce résultat valide l’hypothèse MRU sur l’horizon de 120 s : les aéronefs BlueSky stabilisés volent bien comme le prédicteur le suppose.

Garantie de conflit des exercices. Le générateur construit des paires convergentes à temps d’arrivée égaux (preuve géométrique : $d_{CPA} = 0$ par construction). Sur 2000 tirages seedés, la garantie est vérifiée à 100 % (d_{CPA} maximal 0,082 NM $\ll$ 5 NM de norme de séparation).

5.2 Reconnaissance vocale (STT)

J’ai comparé cinq systèmes sur des échantillons ($n = 150$ par corpus, graine 42, décodage greedy) des deux corpus VHF *réels*, avec mon protocole historique de normalisation et de WER (comparabilité avec les chiffres publiés en S4).

J’en tire trois enseignements (figures 6 et 7) : (i) l’adaptation LoRA est le facteur dominant : le gain sur ATCO2 (domaine jamais vu à l’entraînement) est d’environ 60 % relatif, confirmé par bootstrap apparié ($\Delta \text{WER} = -45$ points, IC 95 % $[-51; -40]$, excluant zéro), très loin devant tout effet de taille de modèle ; (ii) le passe-bande VHF d’inférence améliore ou laisse inchangé le modèle adapté (entraîné dans ce domaine), validant le choix de production ; (iii) le déploiement local est réaliste : tous les systèmes tournent très en deçà du temps réel sur GPU grand public (RTF 0,017

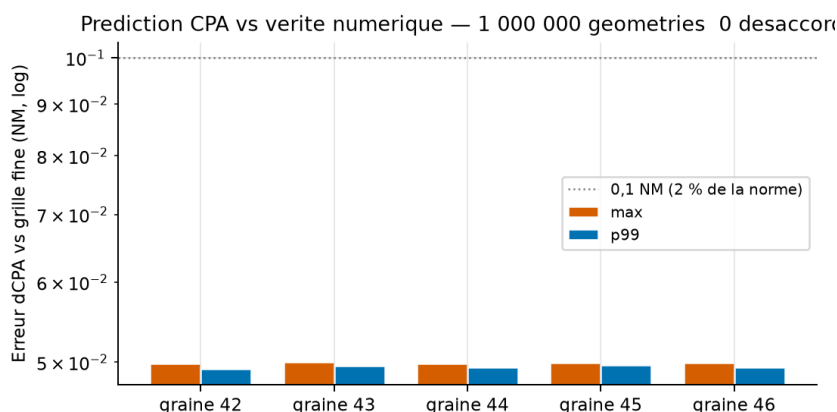


Figure 3 – Erreur de d_{CPA} du prédicteur embarqué face à la grille numérique fine, par graine (200 000 géométries chacune). La décision de conflit est exacte : 0 désaccord sur 10^6 cas.

Système (WER corpus, %)	ATCO2-1h (hors domaine)	UWB-ATCC (test)
faster-whisper tiny	100,9	120,1
faster-whisper base	88,1	107,4
faster-whisper small	71,5	92,3
whisper-small (vanilla)	71,9	91,9
whisper-small + LoRA ATC du projet	28,5	19,2

Table 5 – WER par système sur audio radio réel (condition passe-bande VHF de production pour les systèmes small ; les WER supérieurs à 100 % traduisent un excès d’insertions sur un domaine hors distribution). Détail, IC et conditions complètes : `bench/results/stt_bench.json`.

pour la voie LoRA). La valeur historique de S4 ($74,3 \rightarrow 29,2\%$ sur le test ATCO2 complet) est retrouvée à un point près par la re-mesure indépendante ($71,9 \rightarrow 28,5\%$ sur l’échantillon), ce qui consolide les deux campagnes l’une par l’autre.

5.3 Interprétation des clairances : comparaison appariée de quatre LLM et d’un parseur

Mon corpus d’évaluation compte 116 clairances que j’ai annotées en TrafScript attendu *après validation* : 68 cas dans la grammaire du parseur de référence et 48 cas étendus couvrant paraphrases hors grammaire, bruit de type STT (hésitations, répétitions), français parlé, nombres composés, multi-ordres et cas *mixtes* (ordre valide + ordre invalide dans la même phrase) ; le corpus compte 24 négatifs de sécurité au total. Un cas est réussi si le multi-ensemble d’ordres produit est *exactement* celui attendu. Tous les modèles locaux sont quantifiés Q4_K_M et servis par llama.cpp à température 0.

Système	Global [IC Wilson]	Dans gram.	Hors gram.	Négatifs rejetés	JSON strict	Latence moy.
Parseur à règles (réf.)	87,9 % [80,8 ; 92,7]	100 %	70,8 %	95,8 %	–	~0,05 ms
Mistral-7B-v0.3	81,9 % [73,9 ; 87,8]	85,3 %	77,1 %	75,0 %	100 %	0,82 s
Qwen2.5-1.5B	75,0 % [66,4 ; 82,0]	79,4 %	68,8 %	75,0 %	96,6 %	0,32 s
Qwen2.5-3B	24,1 % [17,3 ; 32,7]	19,1 %	31,3 %	100 %	45,7 %	0,34 s
Qwen2.5-3B (bartowski)	23,3 % [16,5 ; 31,7]	20,6 %	27,1 %	95,8 %	69,0 %	0,42 s
Llama-3.2-1B	3,4 % [1,3 ; 8,5]	1,5 %	6,3 %	16,7 %	22,4 %	2,8 s

Table 6 – Exactitude TrafScript exacte sur les 116 clairances, à travers la chaîne de production complète (prompt + base OACI + validation déterministe). Source : `bench/results/llm_bench.json`.

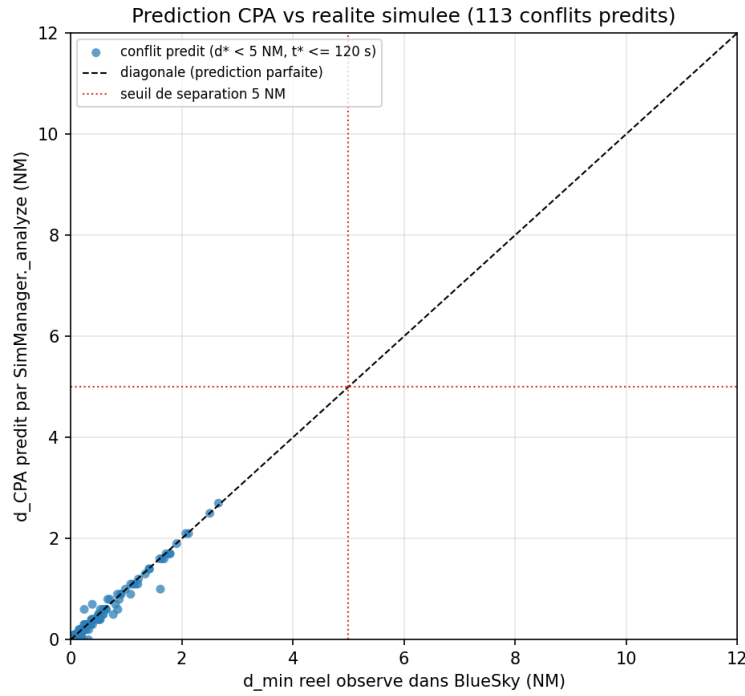


Figure 4 – d_{CPA} prédit contre d_{min} mesuré dans BlueSky (200 rencontres) : MAE 0,067 NM, aucune erreur de classification (TP 113, FP 0, FN 0, TN 87).

La lecture de ce tableau requiert les tests appariés (McNemar exact). Le résultat le plus important est une non-différence : entre le parseur et Mistral-7B, l'écart global n'est pas significatif ($n_{01} = 19$, $n_{10} = 12$, $p = 0,281$). La vraie différence est *structurelle* : le parseur est parfait dans sa grammaire fermée (100 %) et s'effondre dehors (70,8 %), quand le LLM, légèrement moins bon dedans (85,3 %), est *meilleur dehors* (77,1 %) ; c'est exactement l'écart qu'il était chargé de combler, et la raison pour laquelle je garde les deux, le parseur comme référence et oracle de test, le LLM en production. Les autres écarts sont nets : parseur contre Qwen2.5-1.5B, significatif ($p = 0,017$) ; contre Qwen-3B et Llama-1B, massivement significatif ($p < 10^{-20}$). Quant aux deux quantifications du Qwen-3B, elles sont indiscernables ($p = 1,0$) ; c'est le contrôle qui écarte l'artefact de conversion (§4.7).

Sécurité mesurée. Même les petits modèles qui hallucinent des ordres sont rattrapés par la validation déterministe : sur l'ensemble de la campagne, aucun ordre hors bornes ou hors schéma n'a été transmis au simulateur. Le taux de rejet des négatifs par système (tableau 6) chiffre la contribution de chaque couche, illustration directe de l'architecture de la section 3.3.

Point de fonctionnement. Mistral-7B Q4 offre le meilleur compromis qualité/latence locale (81,9 %, 0,8 s) ; étant le modèle servi sur ROMEO en bf16, il fournit aussi la comparaison cluster/local. Qwen2.5-1.5B est le point « basse latence » exploitable (75 %, 0,32 s). Sur la tâche plus ouverte de génération de situations (20 descriptions, 116 contraintes vérifiables), le générateur à règles reste à 100 % contre 71,8 % pour le meilleur LLM : mon choix de production (génération géométrique locale, prouvée ; LLM pour le trafic d'ambiance) est confirmé.

5.4 Synthèse vocale

Pour la synthèse vocale, j'ai fait passer à Kokoro-82M (4 voix) et à Windows SAPI le même exercice, 30 collationnements produits par mon générateur de phraséologie, jugés en coût (RTF) et en *intelligibilité aller-retour* : le texte synthétisé est re-transcrit par un juge STT fixe et le WER (normalisation sémantique symétrique : nombres épelés ↔ chiffres) est mesuré avant et après dégradation VHF. Kokoro synthétise à RTF 0,336 et son intelligibilité aller-retour reste à 21,4 % après canal VHF : la coloration radio coûte peu d'intelligibilité, conformément à l'objectif (réalisme sans perte pédagogique). La métrique partage le biais du juge entre moteurs : elle est **relative**,

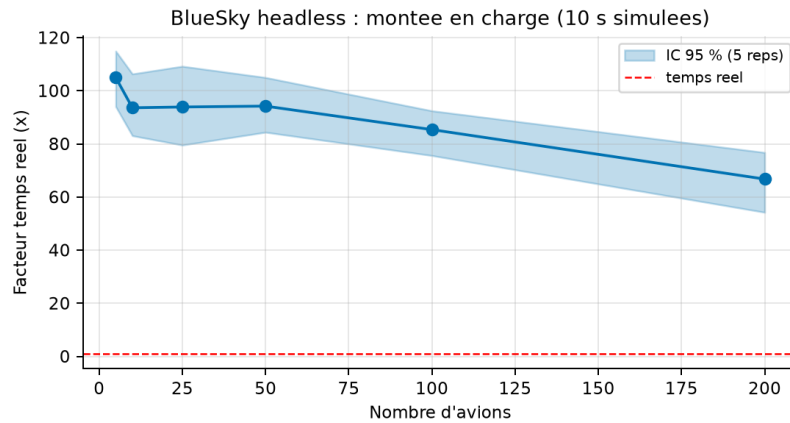


Figure 5 – Montée en charge de BlueSky headless (5 répétitions par point, IC bootstrap 95 %) : $\times 105$ temps réel à 5 aéronefs, $\times 67$ à 200. La simulation n'est jamais le facteur limitant de la boucle vocale.

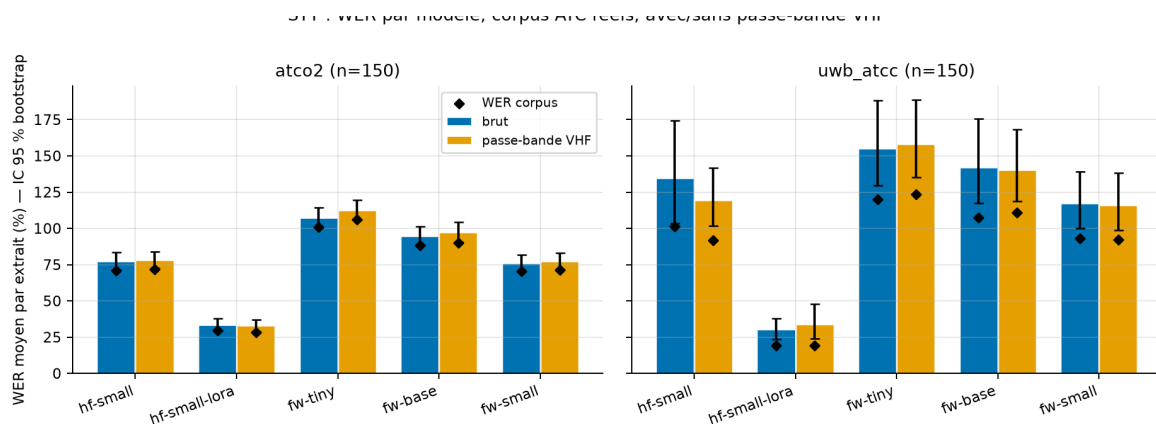


Figure 6 – WER par système et par corpus (IC bootstrap 95 % du WER moyen par extrait ; losanges : WER corpus).

seuls les écarts entre systèmes sont interprétables, limite documentée dans le protocole. Sur le cluster, la référence historique XTTS (voix clonées, CPU) reste 3,9 s par collationnement avec un WER de re-transcription de 22–24 % mesuré par une métrique maison antérieure au banc (non comparable au WER *jiwer*, et signalée comme telle).

5.5 La boucle vocale complète

Ma mesure finale répond à la question d'origine du stage : quand l'élève parle à un avion, celui-ci comprend-il, manœuvre-t-il et répond-il correctement ? Elle ferme la boucle de production entière sur 25 clairances *parlées* : j'ai synthétisé chaque énoncé contrôleur par une voix jamais utilisée ailleurs dans le banc (SAPI), je l'ai fait passer dans un canal radio simulé (bruit SNR 12 dB puis passe-bande VHF), puis la chaîne complète a pris le relais : transcription par mon Whisper-LoRA, interprétation par Mistral-7B, validation, exécution, et collationnement en Kokoro dégradé VHF. Une condition témoin *texte* (mêmes clairances, sans STT) isole le coût de l'étage vocal.

L'écart voix/texte (12 points) chiffre le coût résiduel de la reconnaissance vocale ; la latence totale de 2,6 s est compatible avec le rythme d'une fréquence d'entraînement (un échange toutes les 10–30 s). La figure 12 montre la même boucle vérifiée sur le cluster dès la semaine 8 (descente FL280 → FL180 exécutée puis collationnée en voix clonée).

Contre-épreuve avec un locuteur humain réel. La limite évidente de ce protocole restait la voix synthétique du contrôleur. Pour la lever, j'ai enregistré moi-même les 25 clairances au micro

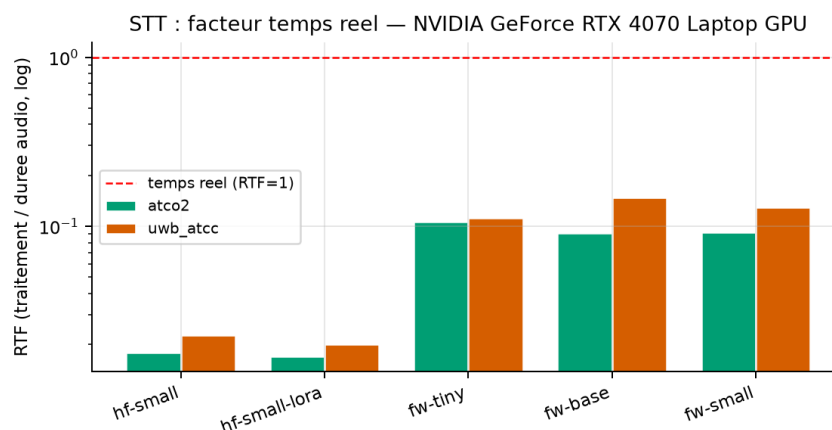


Figure 7 – Facteur temps réel de transcription (échelle log) : tous les systèmes sont largement sous le temps réel sur GPU grand public.

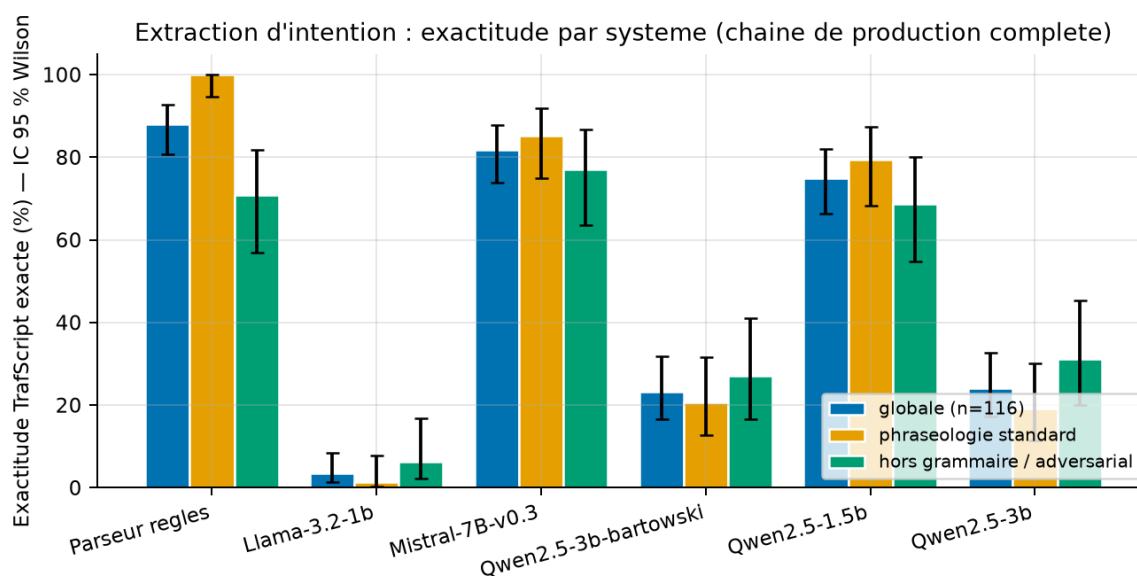


Figure 8 – Exactitude par système (IC Wilson 95 %), globale et par strate : la complémentarité parseur (grammaire fermée) / LLM (hors grammaire) est le résultat structurant.

(capture par navigateur, le même chemin que le push-to-talk de l'application), en phraséologie parlée (indicatifs en téléphonie, chiffres épelés), puis je les ai passées dans la chaîne strictement identique, dans deux conditions : mes prises brutes, telles que l'application m'entend, et les mêmes prises dégradées par le canal radio du banc (bruit SNR 12 dB, même graine). Résultat : 20/25 (80 %, IC 95 % [60,9 ; 91,1]) en condition micro pour 2,3 s de latence moyenne, et 18/25 (72 %, IC [52,4 ; 85,7]) en condition radio, statistiquement indiscernable de la référence à voix synthétique. L'objectif premier du stage, parler aux avions et les entendre répondre, est donc vérifié avec un vrai locuteur, accent français compris. Le détail des échecs est instructif. Ma lecture en téléphonie réussit là où la lecture littérale de la voix SAPI échouait (EZY21, BAW57) ; en revanche l'indicatif « ryanair » est systématiquement déformé par la transcription de ma voix (entendu « win air »), et une clairance « reduce speed two five zero » a été entendue « speed to five zero », produisant un ordre à 50 kt, dans les bornes physiques donc non filtré par les gardes. Le cas le plus révélateur : l'indicatif mixte épelé CSA1DZ est parfaitement transcrit (« csa one delta zulu », WER 0) mais mal réassemblé par le LLM (CSAD12Z), exactement comme avec la voix synthétique ; ce défaut d'assemblage est indépendant du locuteur et désigne une amélioration précise, la normalisation déterministe des indicatifs

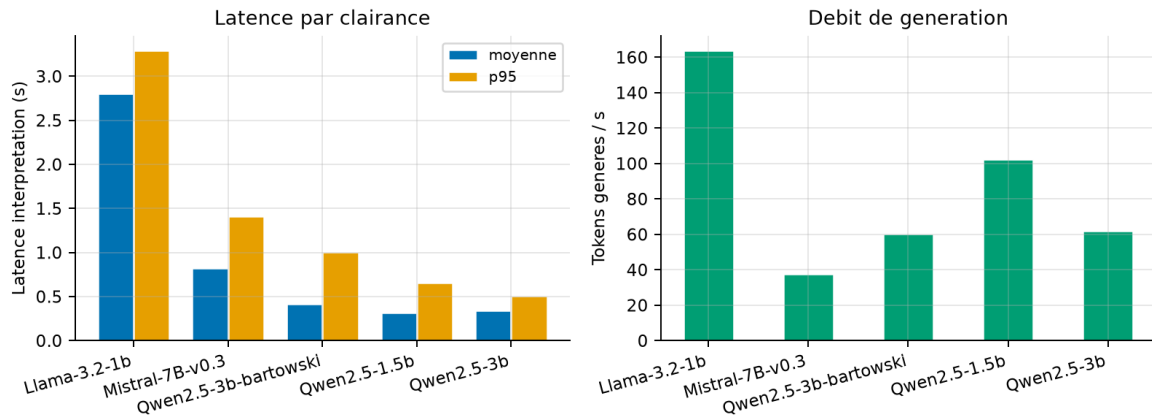


Figure 9 – Latence d’interprétation et débit de génération des modèles locaux (RTX 4070, llama.cpp Q4_K_M, température 0).

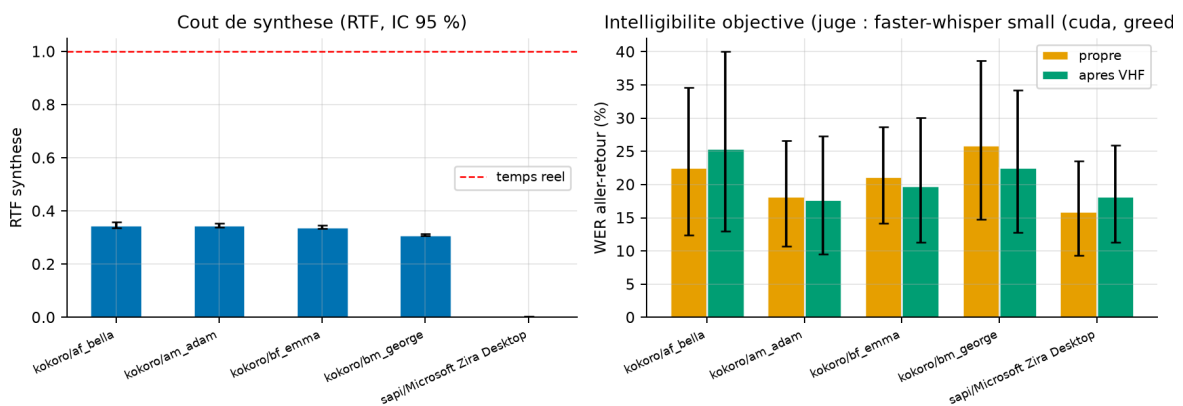


Figure 10 – Synthèse vocale : coût (RTF) et intelligibilité objective aller-retour, avant/après canal VHF.

épelés en amont du modèle. Protocole et résultats versionnés : `bench/human_record_web.py`, `bench/human_e2e.py`, `bench/results/human_e2e.json`.

5.6 Reproductibilité

Reste une question : que valent ces chiffres dans le temps ? Trois dispositifs les empêchent de devenir des instantanés invérifiables : **(i)** j’ai ré-exécuté intégralement la campagne de validation historique le 3 juillet : toutes les métriques sont identiques champ à champ aux JSON versionnés (seules les durées d’exécution diffèrent) ; **(ii)** j’ai muni `validation/run_all.py` de portes de non-régression sur les métriques elles-mêmes (une chute de F_1 casse désormais l’exécution, réponse directe à une lacune relevée par mon audit) ; **(iii)** l’intégralité du banc (scripts, corpus annotés, graines, JSON de résultats, génération des figures et tableaux de l’article) est versionnée dans `bench/`, `validation/` et `docs/article/` du dépôt, et la suite de 209 tests unitaires tourne en intégration continue.

6 Limites et résultats négatifs

Je recense ici, sans les minimiser, les limites et les résultats négatifs du projet, tous documentés dans le dépôt.

Côté modèles, deux échecs nets : Llama-3.2-1B est inutilisable sur la tâche (3,4 %) et Qwen2.5-3B viole le schéma de sortie (§4.7) ; la génération de situations par LLM reste par ailleurs inférieure au générateur géométrique (71,8 % contre 100 %), en particulier sur les contraintes d’espacement. Ces

Boucle complète ($n = 25$)	Condition voix	Témoin texte
Exécutions exactes	76 % (19/25), IC [56,6 ; 88,5]	88 % (22/25), IC [70,0 ; 95,8]
Attribution des 6 échecs voix	3 STT, 3 LLM, 0 fournisseur	–
Latence STT / LLM / TTS (moy.)	0,36 s / 1,06 s / 1,15 s	–
Latence vocale totale	2,6 s (p95 : 3,2 s)	–

Table 7 – Boucle vocale de bout en bout sur GPU grand public (bench/results/e2e_bench.json). Référence cluster historique : ASR 0,84 s, interprétation 5,2 s, TTS 3,9 s.

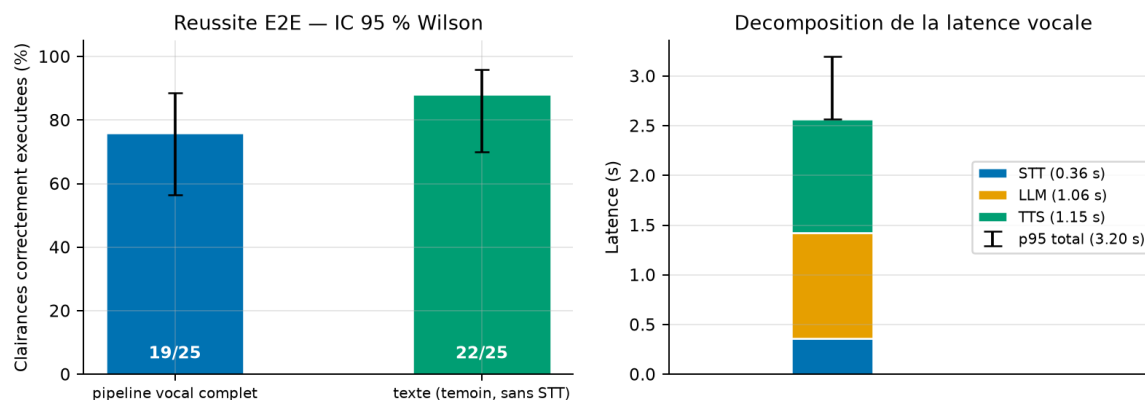


Figure 11 – Boucle vocale complète : taux de réussite (voix contre témoin texte, IC Wilson) et décomposition de la latence par étage.

résultats négatifs sont publiés avec leurs intervalles de confiance et, pour Qwen-3B, avec l'expérience de contrôle.

L'écart voix/texte demeure : 12 points de réussite séparent la boucle parlée du témoin texte ($n = 25$, IC larges), et la moitié des échecs vocaux vient de transcriptions déviées. Le WER de 28,5 % sur radio réelle, quoique divisé par 2,5, reste élevé pour un domaine critique. C'est la validation déterministe qui rend le système utilisable malgré ce taux, en convertissant les erreurs en rejets plutôt qu'en manœuvres erronées.

Mes mesures ont elles-mêmes leurs biais, que je connais. Les énoncés de la référence E2E sont synthétiques (voix SAPI + canal simulé) ; la contre-épreuve à locuteur humain réel (§5.5) borne désormais ce biais, mais elle repose sur un seul locuteur, moi-même, en lecture et non en parole spontanée, sans la phraséologie fluide d'un professionnel ; les corpus réels couvrent par ailleurs ce risque pour l'étage STT isolé. L'intelligibilité TTS est une métrique relative (biais de juge partagé). Le WER historique « 22–24 % » de la boucle XTTS utilisait une métrique maison non comparable à `jiwer`. Le corpus d'interprétation, quoique stratifié et adversarial, reste de taille modeste (116 cas) ; les IC de Wilson en tiennent compte.

La synthèse sur cluster n'est pas temps réel : XTTS reste confiné au CPU par le bug `cuFFT` du GH200 (3,9 s par collationnement), le correctif de compatibilité `transformers` est une dette technique explicite, et le point local (Kokoro, RTF 0,34) contourne le problème mais sans clonage de voix. Le périmètre simulé est lui aussi borné : secteur unique fictif centré sur Reims (7 points), hypothèse MRU sans vent pour le prédicteur (l'application transmet vitesse sol et route, mais la campagne de validation est à vent nul), pas de SID/STAR ni de 4D complet, application mono-poste locale sans authentification (exposition réseau hors périmètre).

Enfin, la traçabilité des chiffres de S4 était partielle : je n'avais pas versionné les artefacts d'entraînement (courbes, `train_summary`), lacune que mon propre audit a relevée. C'est précisément ce qui m'a poussé à re-mesurer le LoRA au banc, de manière indépendante : la re-mesure retrouve



Figure 12 – La boucle vocale sur le cluster (semaine 8) : après une clairance parlée, l’aéronef AFR300 est établi au FL180 et collationne dans sa voix clonée.

les valeurs historiques à un point près et constitue désormais la référence traçable.

7 Bilan

Ce que j’ai démontré à l’issue de ce stage

- **L’objectif premier est atteint : on peut parler aux avions, et ils répondent.** Parole → transcription adaptée au domaine → interprétation ancrée → validation déterministe → manœuvre dans le simulateur → collationnement vocal, à 76 % de réussite en voix synthétique et 80 % en contre-épreuve avec un locuteur humain réel, pour 2,6 s de latence sur GPU grand public, et vérifié sur cluster.
- **Des choix justifiés puis validés par la mesure** : chaque décision structurante (corpus, LoRA, LLM ancré non affiné, BlueSky, contrat OpenAI-compatible) est adossée à l’état de l’art, confrontée à ses alternatives, et confirmée a posteriori par une mesure versionnée (tableau 1).
- **Une base géométrique exacte** : 10^6 décisions de conflit sans désaccord, $F_1 = 1,0$ face à BlueSky, conflits d’exercice garantis à 100 %.
- **Une sécurité qui ne dépend pas du modèle** : sur toute la campagne, aucun ordre hors bornes n’a atteint le simulateur, y compris avec les modèles les plus faibles.
- **Une démarche reproductible** : protocoles seedés, résultats JSON versionnés, ré-exécution identique champ à champ, 209 tests en CI, article de recherche dont chaque chiffre est généré depuis les données brutes.

Sur le plan des compétences, ce stage m’a fait parcourir le cycle complet d’un projet d’IA appliquée : la lecture critique de l’état de l’art et la sélection argumentée (jusqu’à écarter une approche publiée, l’affinage du LLM, pour des raisons de sécurité et de coût) ; l’ingénierie sous contraintes réelles (quota disque, architecture aarch64 exotique, bugs d’écosystème du niveau du gabarit de conversation ou de la DLL) ; la conception d’une architecture de sûreté en couches, et la découverte, par revue adversariale, qu’une de mes propres gardes était contournable ; et la construction d’un appareil de preuve (validation mathématique, campagne empirique, banc

statistique apparié) qui distingue ce qui est démontré de ce qui est simplement plausible.

Ce que ce stage a changé dans ma façon de voir l'IA. Je suis arrivé avec l'idée qu'un bon modèle fait un bon système ; je repars convaincu du contraire. Les épisodes qui m'ont le plus appris sont ceux où le modèle n'était pas en cause : un gabarit de conversation qui perd le message système, une comparaison de chaînes qui rend une garde contournable, un quota disque qui redessine tout un pipeline de données. J'en retire quelques certitudes méthodologiques. Un modèle ne s'évalue qu'à travers sa pile de déploiement réelle, jamais isolément. Dans un domaine où l'erreur a un coût, le déterminisme se place en aval de l'apprentissage : le LLM propose une interprétation, le code la vérifie, chacun fait ce qu'il sait faire. Et la mesure a cessé d'être à mes yeux une formalité de fin de projet ; c'est elle qui a trouvé mes bugs, arbitré mes choix et, au fond, écrit une bonne partie de ce rapport. Ces dix semaines m'ont aussi confirmé un goût personnel : celui de la preuve, plus encore que celui du prototype.

Perspectives. Les suites naturelles sont l'injection de l'état de trafic courant dans la requête unifiée (conscience spatiale située), l'élargissement de la contre-épreuve humaine à plusieurs locuteurs, idéalement des contrôleurs ou élèves contrôleurs en phraséologie spontanée, et l'extension multi-secteurs.

Reproductibilité. Code, corpus annotés, scripts de mesure, résultats bruts et génération des figures : répertoires `bench/`, `validation/` et `docs/article/` du dépôt GitHub du projet (Gotman08/`simulateur-controleur-aerien`).

Références

- [1] J. Zuluaga-Gomez *et al.*, « ATCO2 corpus : A Large-Scale Dataset for Research on Automatic Speech Recognition and Natural Language Understanding of Air Traffic Control Communications », arXiv :2211.04054, 2023.
- [2] L. Šmídl *et al.*, « Air Traffic Control Communication (UWB-ATCC) Corpus », Université de Bohême de l'Ouest, 2019.
- [3] K. Hofbauer, S. Petrik et H. Hering, « The ATCOSIM Corpus of Non-Prompted Clean Air Traffic Control Speech », *LREC*, 2008.
- [4] A. Radford *et al.*, « Robust Speech Recognition via Large-Scale Weak Supervision », *ICML*, 2023.
- [5] E. Hu *et al.*, « LoRA : Low-Rank Adaptation of Large Language Models », *ICLR*, 2022.
- [6] J. L. P. M. van Doorn, *Applying Large-Scale Weakly Supervised Automatic Speech Recognition to Air Traffic Control*, mémoire de master, TU Delft, 2023.
- [7] J. Su et O. Haq, « Fine-Tuning Whisper for American English Air Traffic Control Speech Recognition : A Data-Efficient Pipeline », Research Square, doi :10.21203/rs.3.rs-8970162/v1, 2026.
- [8] Q. Zhang et J. H. Mott, « An Exploratory Assessment of LLM's Potential Toward Flight Trajectory Reconstruction Analysis », arXiv :2401.06204, 2024.
- [9] D. Sid *et al.*, « AirTrafficGen : Configurable Air Traffic Scenario Generation with Large Language Models », arXiv :2508.02269, 2025.
- [10] I. Sharifi, A. Zongo et P. Wei, « Fine-Tuning Large Language Models for Cooperative Tactical Deconfliction of Small Unmanned Aerial Systems », arXiv :2603.28561, 2026.
- [11] J. M. Hoekstra et J. Ellerbroek, « BlueSky ATC Simulator Project : an Open Data and Open Source Approach », *ICRAT*, 2016.
- [12] A. Q. Jiang *et al.*, « Mistral 7B », arXiv :2310.06825, 2023.
- [13] Équipe Qwen, « Qwen2.5 Technical Report », arXiv :2412.15115, 2024.
- [14] E. Casanova *et al.*, « XTTS : a Massively Multilingual Zero-Shot Text-to-Speech Model », *INTER-SPEECH*, 2024.
- [15] OACI, *Doc 4444 : Gestion du trafic aérien (PANS-ATM)*, 16^e édition, 2016. (Source normative ; la base de connaissances du projet est un contenu original, sans extrait verbatim.)